

Normalizing Inputs

1. Purpose

- **Goal:** Normalizing inputs is a technique used to **speed up training**.
- **Effect:** It transforms the training data so that input features are on a similar scale (e.g., all features range roughly from -1 to 1).

2. Implementation Steps

Normalization involves two distinct operations:

1. Subtract the Mean (Zero-Centering):

- Calculate the mean vector μ :
$$\mu = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$
- Update the training set:
$$x := x - \mu$$
- *Result:* The training set has a zero mean.

2. Normalize Variance:

- Calculate the variance vector σ^2 :
$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x^{(i)})^2$$

(Note: Since the mean is already 0, the element-wise square of x represents the variance.)
- Update the training set:
$$x := \frac{x}{\sigma}$$
- *Result:* All features have a variance of 1.

3. Critical Implementation Rule

- **Consistency is Key:** You must use the **same** μ and σ^2 calculated on the **training set** to normalize the **test set**.
- **Do Not:** Calculate a separate mean and variance for the test set.
- **Why?** The model learns parameters based on the specific transformation of the training data. The test data must undergo the exact same transformation to be valid.

4. Intuition: Why does this help?

- **Unnormalized Data (Elongated Cost Function):**
 - If feature scales differ wildly (e.g., $x_1 \in [0, 1000]$ and $x_2 \in [0, 1]$), the cost function $J(w, b)$ becomes distorted.
 - **Shape:** An elongated "bowl" or ellipse.
 - **Gradient Descent:** The algorithm oscillates inefficiently back and forth, requiring a very small learning rate to avoid divergence. Convergence is slow.
- **Normalized Data (Spherical Cost Function):**
 - If all features are on a similar scale, the cost function becomes symmetric.
 - **Shape:** A spherical or round bowl.
 - **Gradient Descent:** The algorithm can take larger steps and move directly toward the minimum without oscillation. Optimization is much faster.

5. When is this necessary?

- **Critical:** When features have dramatically different ranges (e.g., 0 to 1 vs 1 to 1000).
 - **Optional:** When features are already on similar scales (e.g., 0 to 1 vs 1 to 2).
 - **Best Practice:** Since normalization rarely hurts performance, it is often applied by default even if feature scales are similar.
-

Vanishing / Exploding Gradients

1. The Problem

- **Context:** This issue specifically affects **Deep Neural Networks** (networks with a large number of layers, L).
- **Definition:** During training, derivatives (gradients) can become either exponentially big (**Exploding**) or exponentially small (**Vanishing**).
- **Consequence:** This numerical instability makes training difficult.
 - **Vanishing:** Gradient descent takes infinitesimally small steps, making training prohibitively slow.
 - **Exploding:** Weights oscillate significantly or reach numerical overflow (NaN).

2. Mathematical Intuition

To understand why this happens, consider a simplified deep neural network.

- **Assumptions for Simplicity:**

- Linear activation function: $g(z) = z$.
- Bias $b = 0$.
- The output $\hat{y}$ is a product of weight matrices acting on input X :
$$\hat{y} = W^{[L]}W^{[L-1]} \dots W^{[2]}W^{[1]}X$$

Case A: Exploding Gradients

- **Scenario:** Assume weight matrices W are slightly larger than the Identity matrix.

- Example: $W^{[l]} = \begin{bmatrix} 1.5 & 0 \\ 0 & 1.5 \end{bmatrix}$

- **Result:** The output becomes an exponential function of the depth L .
$$\hat{y} = (1.5)^{L-1}X$$
- **Outcome:** As L increases, the activations (and consequently the gradients) grow **exponentially**.

Case B: Vanishing Gradients

- **Scenario:** Assume weight matrices W are slightly smaller than the Identity matrix.

- Example: $W^{[l]} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$

- **Result:** The output decays exponentially.
$$\hat{y} = (0.5)^{L-1}X$$
- **Outcome:** As L increases, the activations decrease **exponentially** toward zero.

3. Impact on Training

- **Historical Barrier:** This phenomenon was a major obstacle that prevented deep learning from working effectively for a long time.
 - **Modern Context:** With extremely deep networks (e.g., ResNets with 152 layers), even small deviations from 1 in the weights can compound into massive or tiny values.
 - **Partial Solution:** While this problem doesn't have a complete fix, it can be significantly mitigated through **careful weight initialization**.
-

Weight Initialization for Deep Networks

1. Motivation

- **Problem:** Deep neural networks suffer from **Vanishing** or **Exploding Gradients**, where derivatives become exponentially small or large as they backpropagate through layers.
- **Solution:** While not a complete cure, careful initialization of weights significantly mitigates this problem by ensuring the scale of activations (a) and gradients remains consistent throughout the network.

2. Intuition: Single Neuron

Consider a single neuron with inputs $x_1, \dots, x_n$ and weights $w_1, \dots, w_n$.

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

(ignoring b for now)

- **Variance Analysis:**
 - If n (number of input features) is large, the sum z will naturally become large if weights w are constant.
 - To keep z within a reasonable scale (similar to the inputs), the weights w_i must become smaller as n increases.
 - **Goal:** We want the variance of the output z to match the variance of the input x .
 - **Mathematical Result:** To achieve this, the variance of the weights w should be inversely proportional to n .
$$\text{Var}(w) \approx \frac{1}{n}$$

3. Initialization Formulas

The specific constant used depends on the **activation function** applied in the layer.

Let $n^{[l-1]}$ be the number of inputs to layer l (also known as fan-in).

A. For ReLU Activation (He Initialization)

Because ReLU zeroes out half the inputs (negative values), the variance needs to be doubled to maintain the signal's magnitude.

- **Recommended Variance:** $\frac{2}{n^{[l-1]}}$
- **Implementation (Python/NumPy):**

$$W^{[l]} = \text{np.random.randn}(\dots) \times \sqrt{\frac{2}{n^{[l-1]}}}$$

- *Note: This is considered the standard best practice for ReLU networks (Paper by He et al., 2015).*

B. For Tanh Activation (Xavier Initialization)

- **Recommended Variance:** $\frac{1}{n^{[l-1]}}$

- **Implementation:**

$$W^{[l]} = \text{np.random.randn}(\dots) \times \sqrt{\frac{1}{n^{[l-1]}}}$$

- **Alternative (Xavier/Glorot):** Some papers suggest using the average of input and output connections:

$$\sqrt{\frac{2}{n^{[l-1]} + n^{[l]}}}$$

4. Implementation & Tuning

- **Process:** Generate random numbers from a Gaussian (Normal) distribution (mean 0, variance 1) and then multiply by the scaling factor derived above.
- **Hyperparameter Status:**
 - The multiplicative term (e.g., the "2" in He initialization) can effectively be treated as a hyperparameter.
 - **Priority:** It is generally a lower priority hyperparameter. Tuning it usually has a modest effect compared to learning rate or network architecture, but it can sometimes help.

Numerical Approximation of Gradients

1. Purpose

- **Goal:** Before implementing full "Gradient Checking," we need a method to numerically approximate the derivative (gradient) of a function.
- **Why?** Backpropagation involves complex calculus and coding. It is easy to introduce subtle bugs. A numerical approximation provides a "sanity check" to verify that your efficient (but complex) analytical gradient code is correct.

2. One-Sided vs. Two-Sided Difference

To approximate the gradient of a function $f(\theta)$ at a point θ :

A. One-Sided Difference (Less Accurate)

- **Formula:**
$$\frac{f(\theta+\epsilon)-f(\theta)}{\epsilon}$$
- **Analogy:** Calculating the slope using the current point and a point slightly to the right.
- **Accuracy:** The error is $O(\epsilon)$ (order of epsilon). If $\epsilon = 0.01$, the error is roughly 0.01.

B. Two-Sided Difference (More Accurate)

- **Formula:**
$$\frac{f(\theta+\epsilon)-f(\theta-\epsilon)}{2\epsilon}$$
- **Analogy:** Calculating the slope using a point slightly to the right and a point slightly to the left.
- **Accuracy:** The error is $O(\epsilon^2)$ (order of epsilon squared). If $\epsilon = 0.01$, the error is roughly 0.0001.
 - *Note:* Since $\epsilon < 1$, ϵ^2 is much smaller than ϵ , making this method significantly more precise.

3. Example Calculation

Let $f(\theta) = \theta^3$. We want to find the gradient $g(\theta)$ at $\theta = 1$.

- **Analytical Gradient:** $g(\theta) = 3\theta^2$. At $\theta = 1$, $g(1) = 3$.
- **Numerical Approximation:**
 - Let $\epsilon = 0.01$.
 - $\theta_{right} = 1.01$, $\theta_{left} = 0.99$.
 - Calculation:
$$\frac{(1.01)^3 - (0.99)^3}{2(0.01)} = \frac{1.030301 - 0.970299}{0.02} = \frac{0.060002}{0.02} = 3.0001$$
 - **Error:** $3.0001 - 3 = 0.0001$. This is extremely close to the true value.

4. Conclusion for Gradient Checking

- **Trade-off:** The two-sided difference method requires evaluating the function twice ($f(\theta + \epsilon)$ and $f(\theta - \epsilon)$), making it roughly **twice as slow** as the one-sided method.
- **Verdict:** For **Gradient Checking** (which is done only for debugging, not

training), the increased accuracy is worth the computational cost. Always use the **Two-Sided Difference** formula.

Gradient Checking ("Grad Check")

1. Purpose

- **Goal:** Gradient checking is a debugging technique used to verify the correctness of a backpropagation implementation.
- **Problem:** Backpropagation code is complex and prone to subtle bugs (e.g., off-by-one errors, sign errors) that may not cause the code to crash but will result in a network that learns poorly.
- **Solution:** Compare the analytical gradients (computed via backpropagation) with numerically approximated gradients.

2. Implementation Steps

Step 1: Reshape Parameters and Gradients

Convert all parameter matrices ($W^{[l]}, b^{[l]}$) and their corresponding gradients ($dW^{[l]}, db^{[l]}$) into two giant vectors.

- θ (**Theta**): Concatenate all W and b parameters into a single large vector.
 - $J(W, b) \rightarrow J(\theta)$
- $d\theta$: Concatenate all dW and db derivatives into a single large vector of the same dimension as θ .

Step 2: Compute Numerical Approximation

Loop through each component i of the vector θ to calculate the approximate gradient $d\theta_{\text{approx}}[i]$ using the **Two-Sided Difference** formula:

$$d\theta_{\text{approx}}[i] = \frac{J(\theta_1, \dots, \theta_i + \epsilon, \dots) - J(\theta_1, \dots, \theta_i - \epsilon, \dots)}{2\epsilon}$$

- ϵ : A very small number (typically 10^{-7}).
- **Result:** A vector $d\theta_{\text{approx}}$ that should be extremely close to the vector $d\theta$ derived from backpropagation.

Step 3: Compare Vectors

Calculate the distance between the two vectors $d\theta_{\text{approx}}$ and $d\theta$ to quantify their difference. The standard metric uses the Euclidean Norm ($\|\cdot\|_2$):

$$\text{Difference} = \frac{\|d\theta_{\text{approx}} - d\theta\|_2}{\|d\theta_{\text{approx}}\|_2 + \|d\theta\|_2}$$

- **Numerator:** The Euclidean distance between the vectors.
- **Denominator:** Normalizes the value based on the lengths of the vectors, preventing issues if the gradients are extremely large or small.

3. Interpreting the Result

Using $\epsilon = 10^{-7}$, use the following thresholds to evaluate the implementation:

Difference Value	Diagnosis	Action
$< 10^{-7}$	Great	The implementation is very likely correct.
$\approx 10^{-5}$	Acceptable / Warning	Double-check the components. If some specific $d\theta[i]$ values have large differences, there may be a bug.
$\geq 10^{-3}$	Fail / Bug Likely	There is almost certainly a bug in the backpropagation code. Inspect the individual components of θ to find which specific derivative is failing.

Gradient Checking Implementation Notes

1. Performance Warning

- **Rule:** Do not use gradient checking during training. Use it **only** for debugging.
- **Reason:** Gradient checking is extremely slow.
 - To approximate the gradient for *every* parameter, you must evaluate the cost function J twice (for $\theta + \epsilon$ and $\theta - \epsilon$).
 - In a deep network with millions of parameters, this is computationally infeasible for every iteration of Gradient Descent.

- **Workflow:**

1. Implement Backprop.
2. Run Gradient Check to verify correctness.
3. **Turn off** Gradient Check.
4. Run the training loop using standard Backprop.

2. Identifying Bugs by Component

If the Gradient Check fails (difference $> 10^{-3}$), use the vector components to diagnose the specific location of the bug.

- **Mechanism:** Compare the vectors $d\theta_{\text{approx}}$ and $d\theta$ element by element.
- **Diagnosis Examples:**
 - **Isolate b vs W :** If the large differences are all indices corresponding to db (biases) and the dW (weights) indices are correct, the bug is likely in the bias update or gradient calculation.
 - **Isolate Layers:** If the errors are clustered in indices corresponding to a specific layer (e.g., Layer $l = 3$), focus debugging efforts on the forward/backward propagation implementation for that specific layer.

3. Handling Regularization

- **Requirement:** If your model uses L2 Regularization, the Cost Function $J(\theta)$ includes the penalty term $(\frac{\lambda}{2m} ||W||^2)$.
- **Implementation:** Ensure that your numerical approximation of $J(\theta)$ **includes this regularization term**.
- **Gradients:** Similarly, ensure your analytic $d\theta$ (from backprop) accounts for the regularization derivative $(\frac{\lambda}{m} W)$.

4. Incompatibility with Dropout

- **Problem:** Gradient Checking does **not** work with Dropout.
 - Dropout randomly changes the network architecture (and thus the cost function J) on every iteration.
 - It is difficult to define a consistent J to numerically approximate.
- **Solution:**
 1. Set `keep_prob = 1.0` (Turn off Dropout).
 2. Run Gradient Check to verify the base algorithm is correct.
 3. Turn Dropout back on (reset `keep_prob`) for training.

5. Random Initialization Timing

- **Subtle Bug:** It is possible (though rare) for an implementation to be correct when weights W , b are close to 0 (initialization) but incorrect when they become large.
 - **Strategy:**
 1. Run Gradient Check at **random initialization**.
 2. Train the network for a short time (allow W , b to grow).
 3. Run Gradient Check **again** to ensure the implementation holds for non-zero parameters.
-

Week 1 Summary

Congratulations on completing the first week of "Improving Deep Neural Networks." Here is a recap of the key skills you have acquired:

- **Setting up ML Applications:**
 - Splitting data into **Train / Dev / Test** sets.
 - Understanding the shift in split ratios for **Big Data**.
 - **Diagnostics:**
 - Analyzing **Bias vs. Variance** using Train and Dev error.
 - The "Basic Recipe" for fixing High Bias (bigger network) and High Variance (more data/regularization).
 - **Regularization:**
 - **L2 Regularization:** Penalizing large weights ($\|W\|_F^2$).
 - **Dropout:** Randomly removing units to force feature redundancy.
 - **Data Augmentation & Early Stopping.**
 - **Optimization Setup:**
 - **Normalizing Inputs** to speed up convergence.
 - **Vanishing/Exploding Gradients:** The problem of deep networks.
 - **Weight Initialization:** Using He/Xavier initialization to mitigate gradient issues.
 - **Debugging:**
 - **Gradient Checking:** Numerically verifying your backpropagation code.
-